

Dagster at the center of your pipelines

WPipe vs. Dagster: Is "Asset-Based" Always Best?

The context

Dagster has done great work with its asset-based approach, but at what cost? For many projects, the infrastructure needed to run Dagster is simply too heavy. 🦾 WPipe offers a "Code-First" alternative that prioritizes lightness and resilience without the need for complex IO Managers.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="step", version="v2.1")
def step(data):
    return {"value": process(data["row"])}

pipe = Pipeline(pipeline_name="dagster_like")
pipe.set_steps([step])

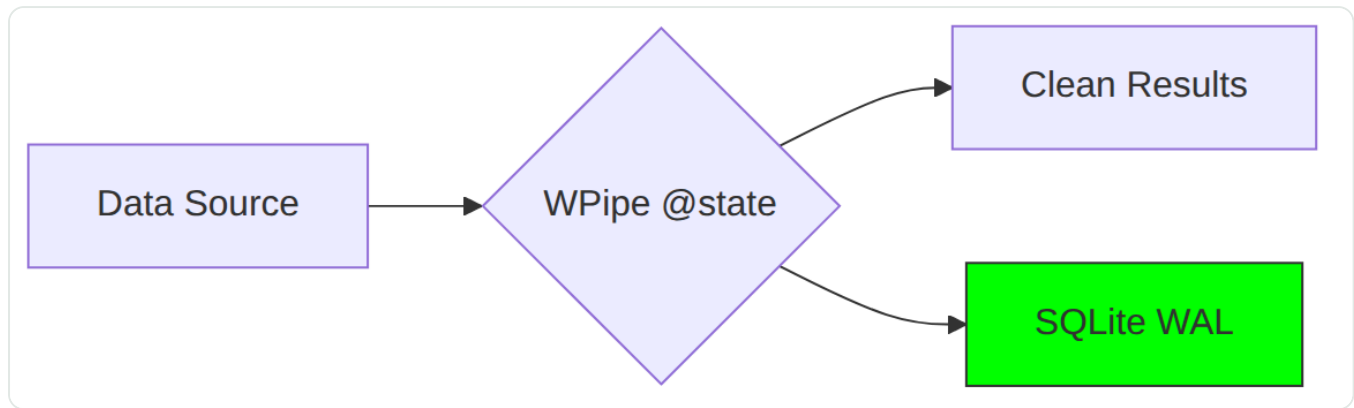
# Step-first resilience without the server.
```

At a glance

Feature	WPipe	Dagster
RAM Usage	< 50MB	> 500MB
Setup	Instant	Moderate
Resilience	SQLite WAL	DB / IO Managers
Edge Ready	✅ Yes	❌ Difficult

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Dagster** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.